

Step 2: 图片懒加载 (loading="lazy")

2.1 在 index.html 的项目卡片图片标签上添加属性

假设你的项目卡片里有图片 (如果目前没有图片, 可以先在 `.project-card` 里加一个 `<img>` 标签练习):

```
html

<article class="project-card">
  
  <h3>Project 1</h3>
  <p>项目描述...</p>
</article>
```

2.2 逐行解读

代码	含义
<code></code>	普通的图片标签, <code>src</code> 指定图片路径
<code>alt="Project 1 截图"</code>	图片的替代文字, 图片加载失败或屏幕阅读器朗读时会用到 (这是 Day 1 学过的无障碍知识点, 今天顺带复用)
<code>loading="lazy"</code>	浏览器原生支持的懒加载属性: 图片只有滚动到接近可视区域时才会真正开始加载

2.3 代码中完整的名称与关系列表

按代码出现顺序, 逐个交代"它是什么身份、和谁发生什么关系":

出现顺序	名称	它是什么身份	和谁发生什么关系
1	<code></code>	HTML 标签	声明页面上有一张图片，是懒加载作用的对象
2	<code>src</code>	<code></code> 的属性	指定图片文件的路径，浏览器根据这个地址去请求图片资源
3	<code>alt</code>	<code></code> 的属性	与 <code>src</code> 并列，提供图片的文字替代内容，和加载时机无关，是无障碍/SEO 属性
4	<code>loading</code>	<code></code> 的属性	控制浏览器何时发起 <code>src</code> 指向的图片请求，值为 <code>"lazy"</code> 时推迟加载
5	浏览器的懒加载机制	浏览器内置行为（不是你写的代码，而是浏览器解析 <code>loading="lazy"</code> 后自动执行的逻辑）	持续监测这张图片是否接近可视区域，一旦接近就自动触发 <code>src</code> 请求

2.4 代码结构解析

text

```

```

浏览器解析时的处理逻辑：

```
|
├─ 页面首次加载
│   └─ 【判断节点】这张 <img> 当前是否在可视区域附近？
│       ├── 是 → 立即发起 src 请求，正常加载
│       └─ 否 → 【推迟节点】暂不请求，只占位（不影响布局）
└─ 用户滚动页面
    └─ 【触发节点】图片进入可视区域附近
        └─ 自动发起 src 请求，图片开始加载
```

关键节点说明：`loading="lazy"` 不改变图片"最终会显示"这件事，只改变"什么时候开始请求"——这正是性能优化的核心思路：不是不做，而是"把不紧急的事情往后推"，优先保证首屏尽快可用。

2.5 三种 loading 属性值对比

值	效果	适合场景
<code>loading="eager"</code> （默认值，等同不写）	立即加载，不管是否在可视区域	首屏就能看到的重要图片（比如 Hero 区域的主图）
<code>loading="lazy"</code>	推迟到接近可视区域才加载	页面下方、需要滚动才能看到的图片（比如项目卡片列表）
不加 <code>loading</code> 属性	等同于 <code>eager</code> ，所有图片一开始就全部加载	—

2.6 生活化类比

`loading="lazy"` 就像**自助餐厅的分批上菜**：不会把所有菜一次性摆满整张桌子（那样很多菜会凉掉、占地方），而是先上你面前马上要吃的（首屏图片），后面的菜等你吃到那个位置附近，服务员才端上来（滚动到附近才加载）——既保证了效率，也不浪费资源。

2.7 完成标准

- 1. 保存 `index.html`
- 2. 打开浏览器开发者工具 → Network 面板，筛选 "Img"
- 3. 刷新页面（不滚动），再慢慢向下滚动

你应该看到：刚刷新页面时，Network 面板里下方卡片的图片**还没有**出现在请求列表里；随着你往下滚动，图片请求才一个个出现。

完成后请回复：

- "验证通过，进入下一步"
- 或 "遇到问题：____"